

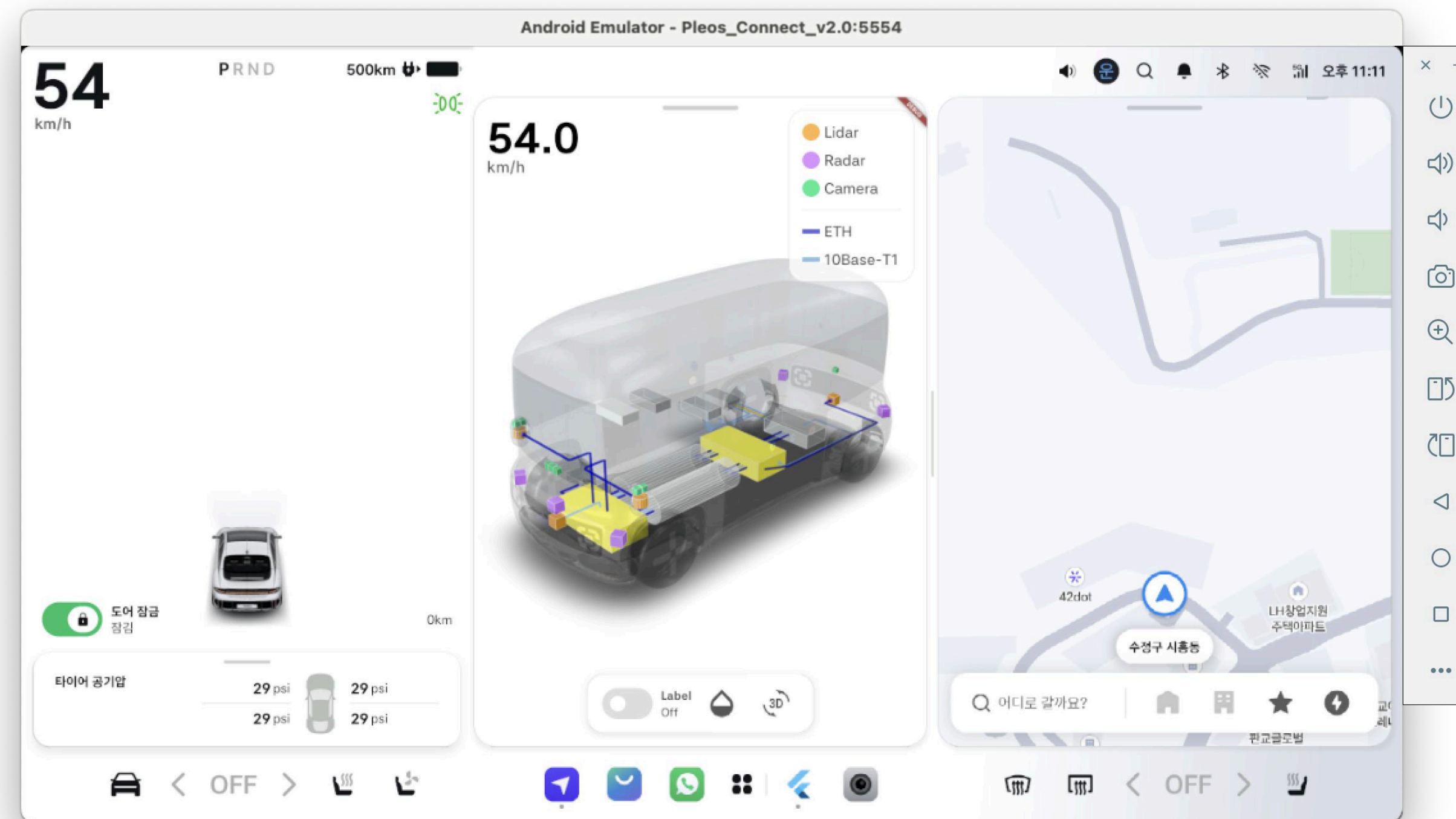
Pleos Connect 기반 Zonal 아키텍처 응용 및 관제 SW 발표 및 데모

김정원 연구원, 2025.12.11

1. 소프트웨어 전체 개요

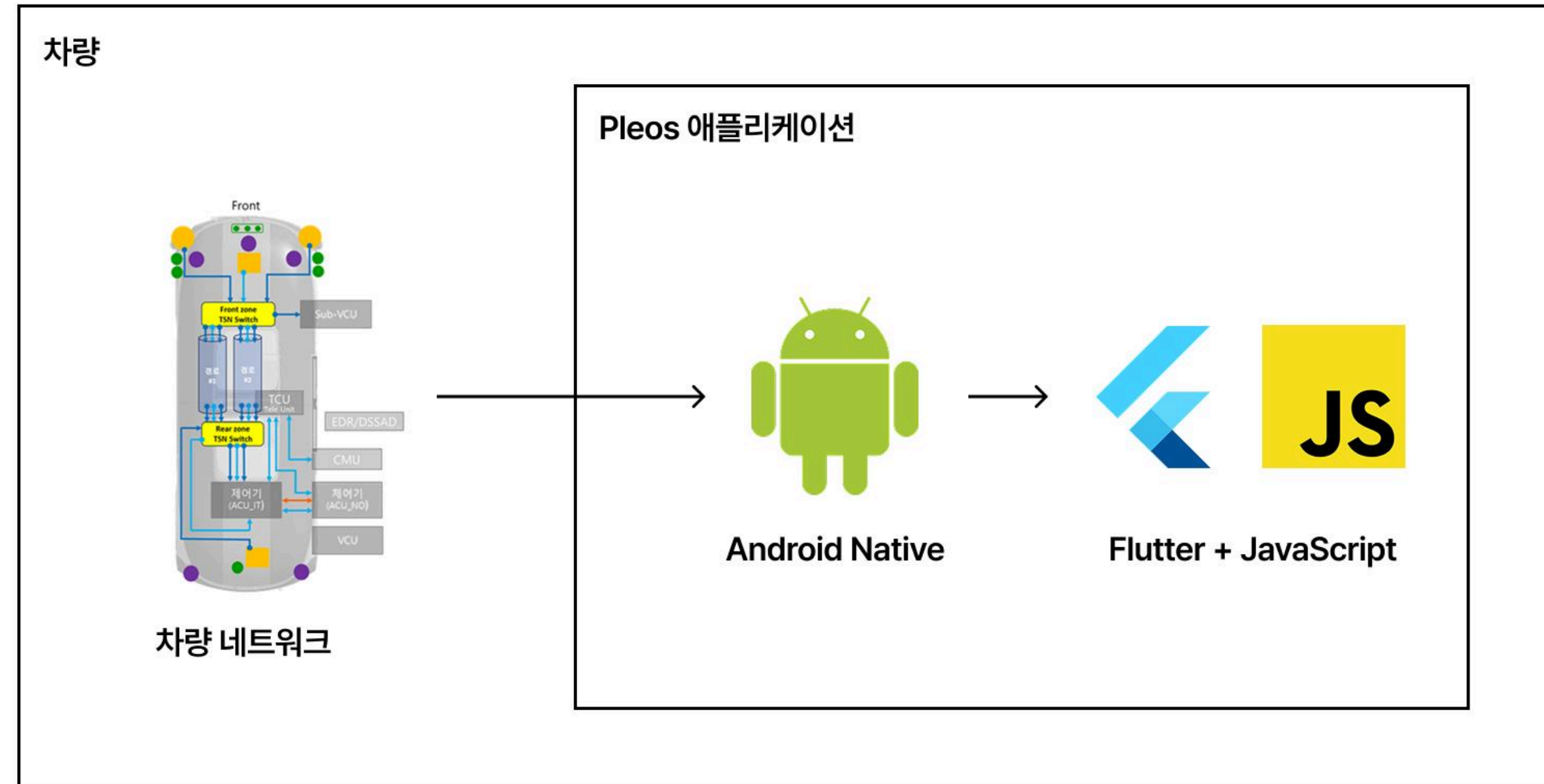
개발 내용 : Pleos Connect(AAOS) 환경에서 동작하는 Zonal 아키텍처 응용 및 관제 SW 로, 차량의 복잡한 Zonal 네트워크(TSN)에서 발생하는 고장 상황을 3D 차량 모델 상에 직관적으로 시각화한다.

개발 목적 : 개발자 및 엔지니어가 고장 원인을 신속하게 파악하고 대응 시나리오를 검증할 수 있도록 지원한다.



2. 시스템 아키텍처

2.1. 전체 구조도



- 계층 분리
 - 안드로이드 네이티브: 통신 프로토콜 처리 및 데이터 파싱
 - 플러터: 데이터 매핑 및 UI 렌더링
- [차량 네트워크] → [안드로이드 네이티브] → [플러터]로 이어지는 단방향 흐름

2. 시스템 아키텍처

2.2. 데이터 흐름 - 데이터 형태 및 매핑 전략

페이로드의 크기를 줄여 성능을 확보하고, 유지보수를 용이하게 하기 위해 다음과 같이 데이터 형식 정의

외부 수신 데이터(Raw Data, JSON)

```
{
  "action": 1,
  "id": 1234,      // Unique ID
  "code": 101,     // Fault Code
  "target": "RearZC",
  "severity": 2
}
```

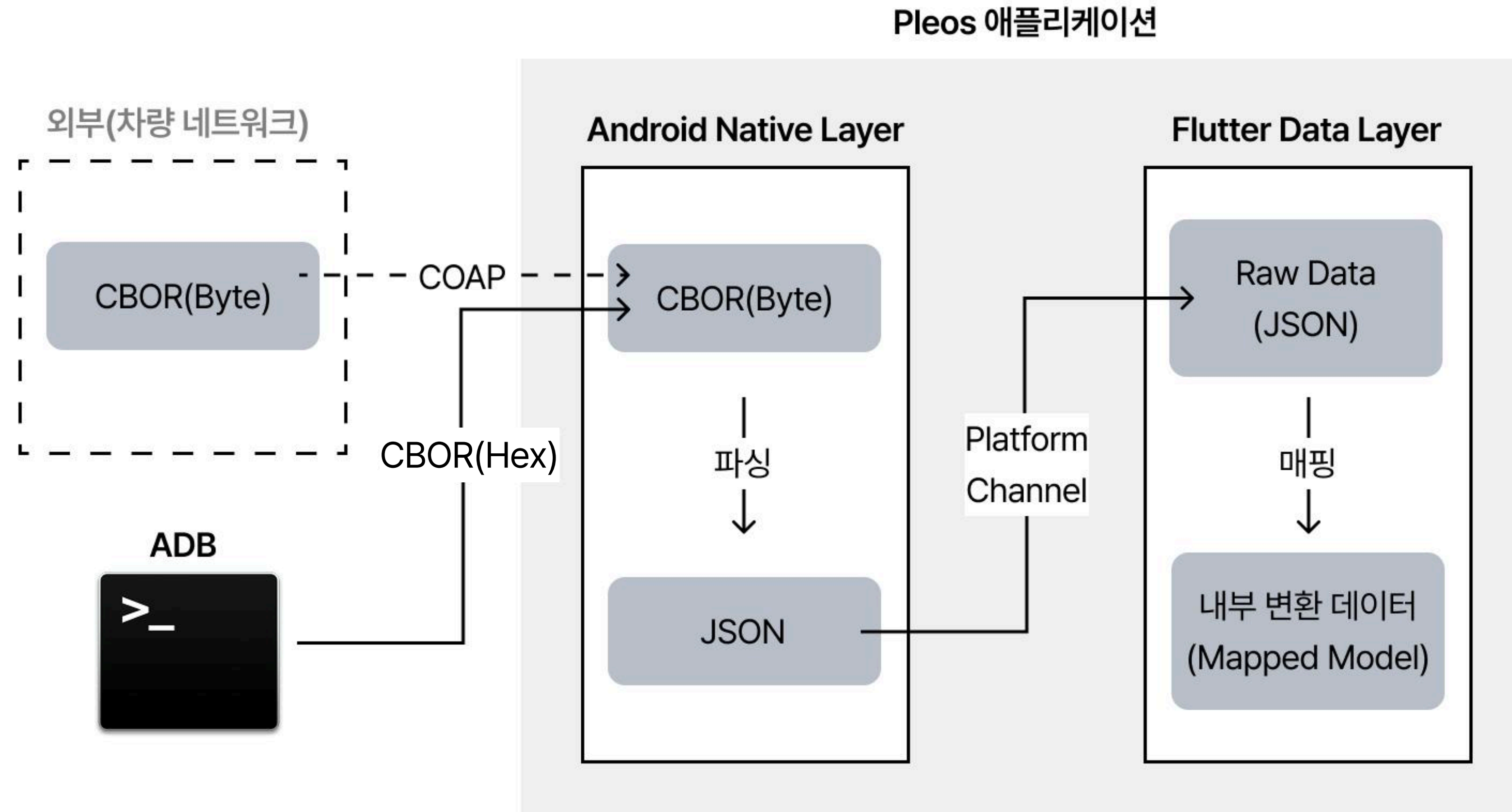


내부 변환 데이터(Mapped Model, Dart)

```
FaultData(
  id: 1234,
  target: "RearZC",
  severity: 2,
  faultType: "시간 동기 상실",
  cause: "PTP 메시지 전파 경로 상 링크 단절",
  countermeasures: [
    "PTP용 FRER 설정",
  ],
)
```

2. 시스템 아키텍처

2.2. 데이터 흐름 - 데이터 파이프라인

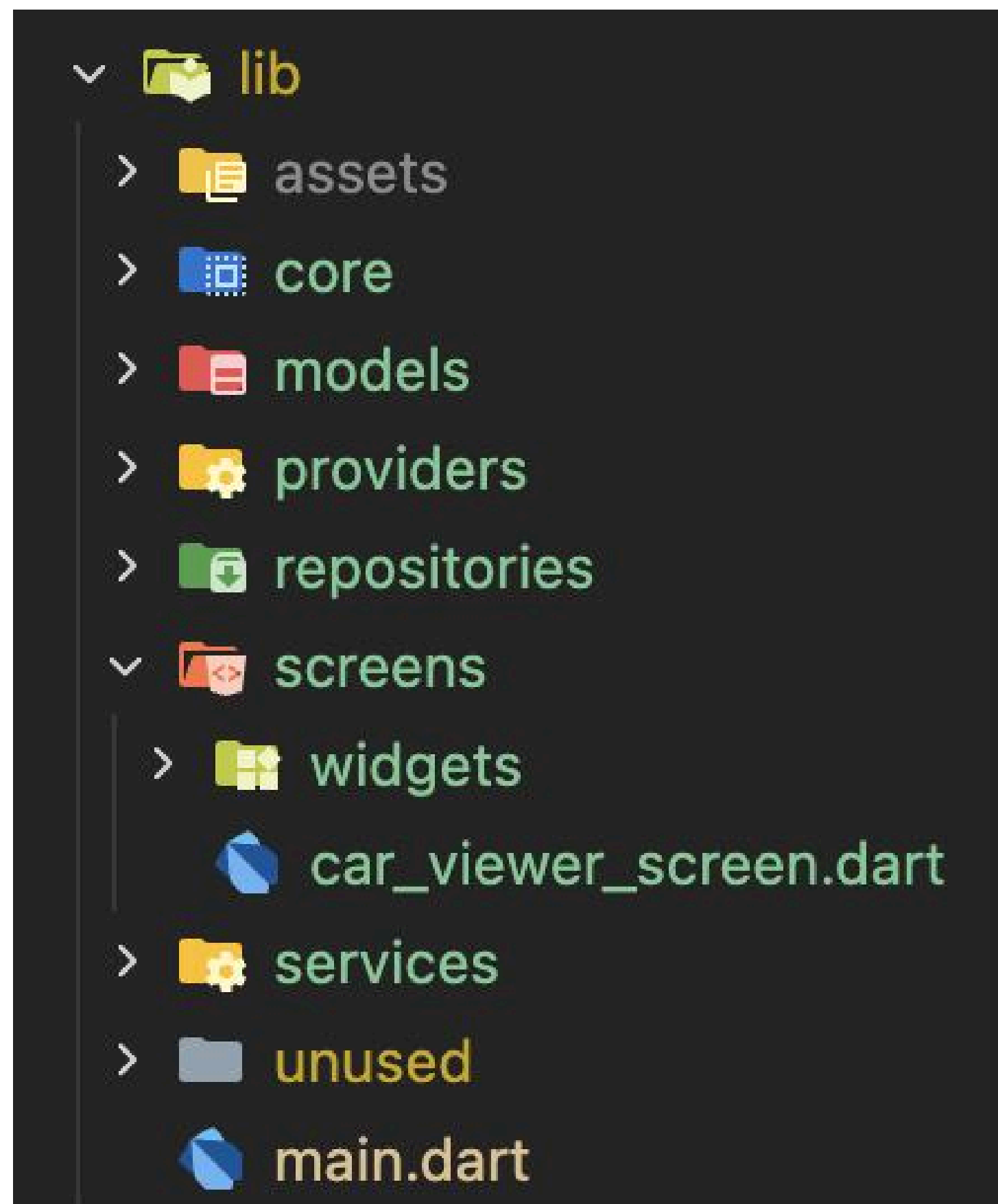


ADB(Android Debug Bridge): 컴퓨터와 Android 기기를 연결해 서로 통신하고 제어할 수 있도록 해주는 프로그램

3. 구현 설명

3.1. 코드 - 플러터

유지보수를 용이하게 하고, 코드 간 의존성을 줄이기 위해 MVVM(Model-View-ViewModel) 아키텍처 사용

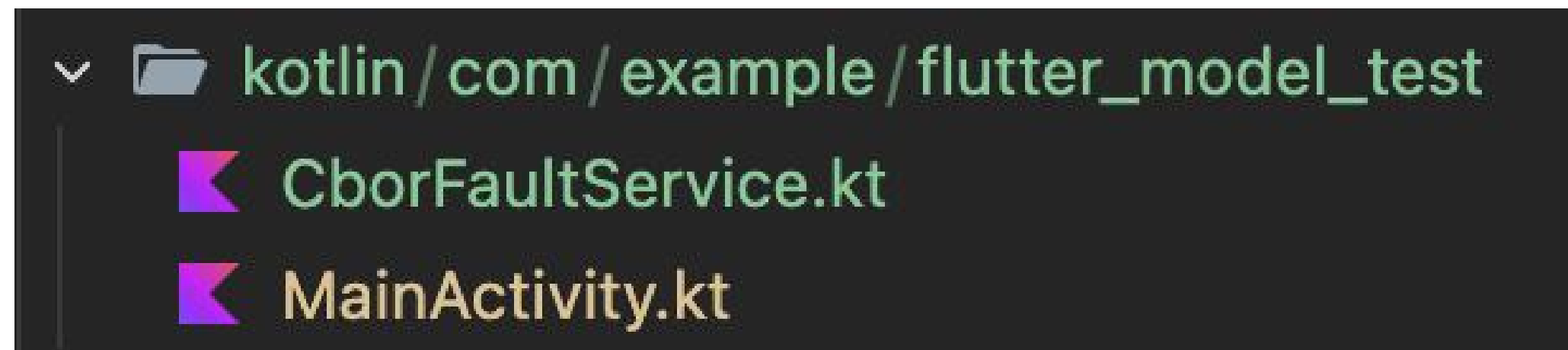


1. View: 화면 렌더링(UI) 담당
 - screens/ - 앱의 각 페이지
 - widgets/ - 재사용 가능한 UI 컴포넌트
2. ViewModel: Model의 데이터를 가공하여 View가 사용할 수 있는 상태(State)로 관리
 - providers/ - 고정 데이터의 상태 관리
3. Model: 데이터 구조 정의 및 데이터 수신과 가공
 - models/: 데이터 객체의 구조 정의
 - services/: 외부 시스템(안드로이드 네이티브)와의 통신
 - repositories/: 데이터 변환

3. 구현 설명

3.1. 코드 - 안드로이드 네이티브

통신 프로토콜 처리 및 데이터 파싱 담당



1. MainActivity.kt: 플랫폼 채널 관리

- 권한 요청 - AAOS 시스템 권한 요청/승인 처리
- 차량 속도 스트림 - 실시간 속도 변화 전달
- 고장 데이터 스트림 - CborFaultService가 감지한 Zonal 아키텍처 고장 데이터 실시간 전달

2. CborFaultService.kt: Zonal 아키텍처 고장 데이터 처리

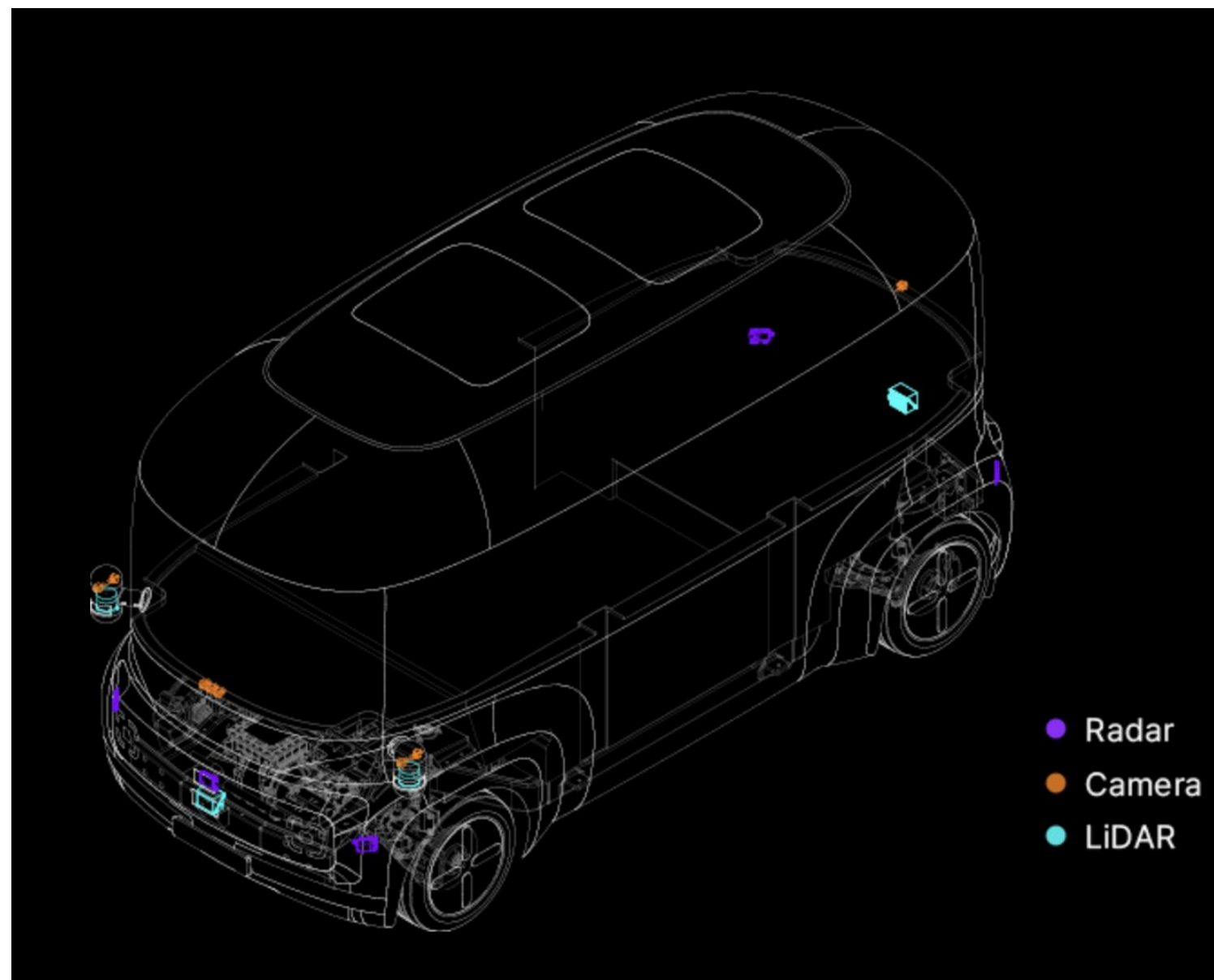
- CBOR 데이터 수신
- CBOR → JSON 변환 후 Flutter로 전송

플랫폼 채널(Platform Channel): UI 렌더링을 담당하는 Flutter가 안드로이드 네이티브의 하드웨어/OS 기능에 접근하기 위한 통신 브릿지

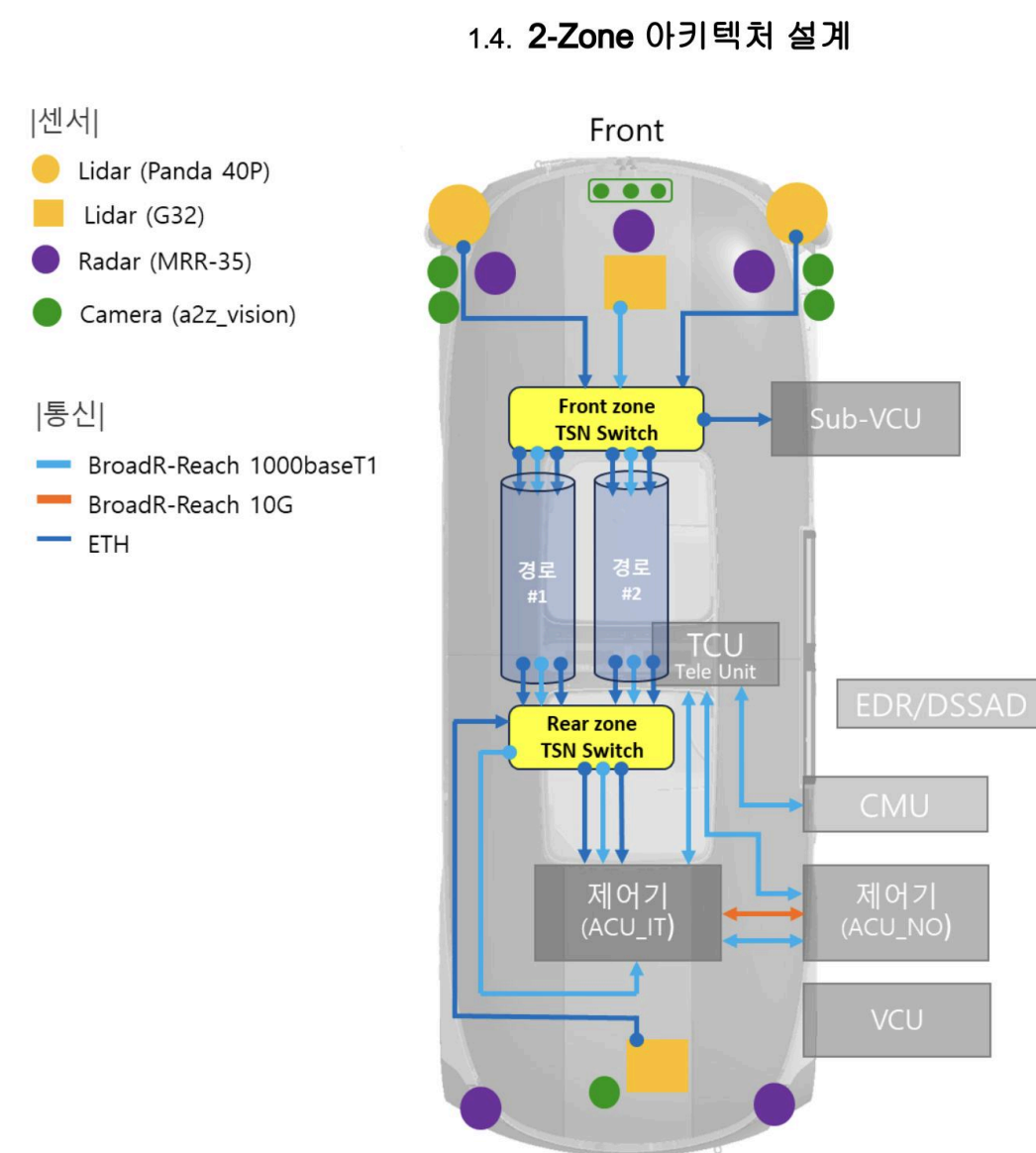
3. 구현 설명

3.2. 3D 모델링 및 시각화

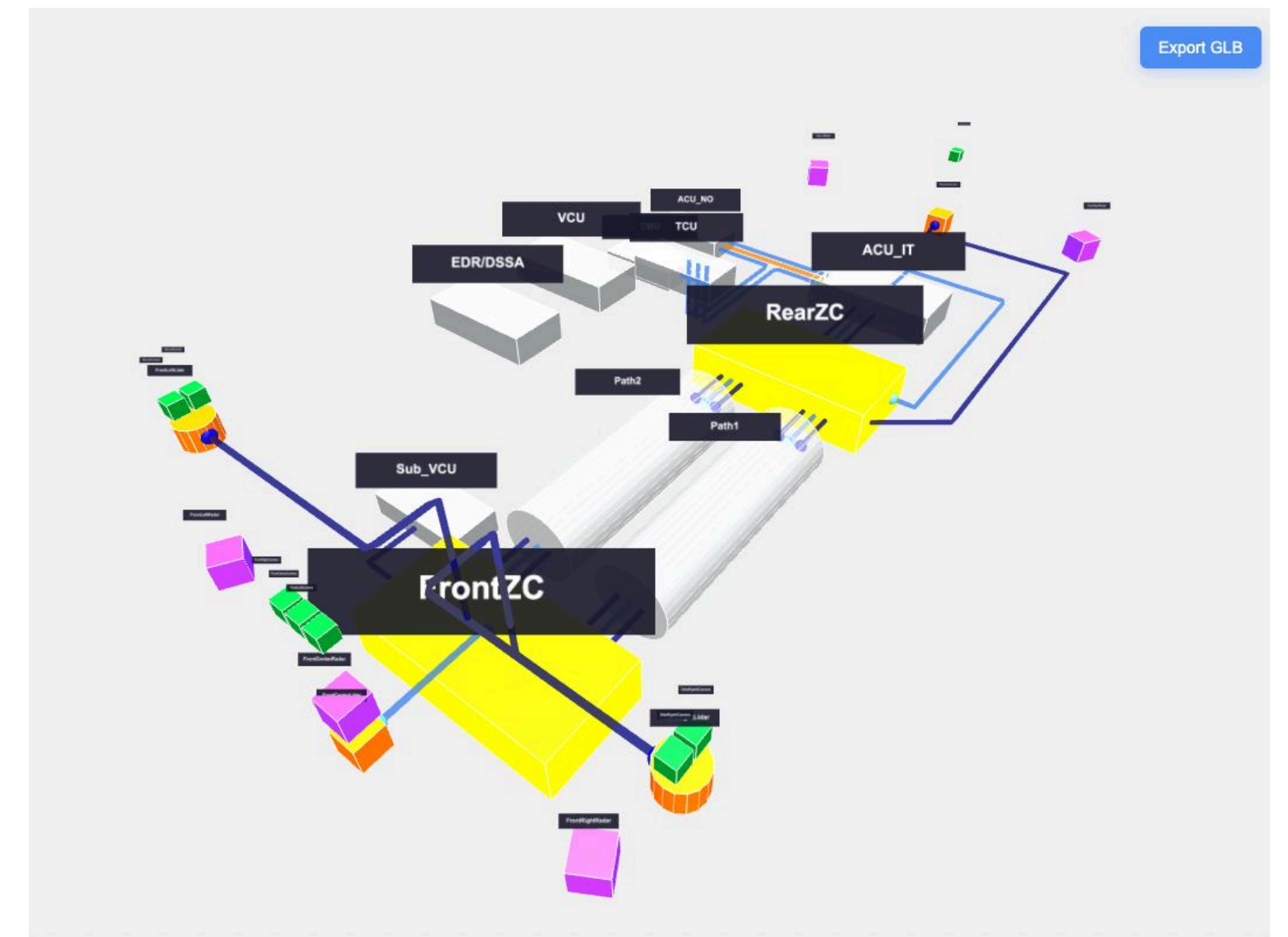
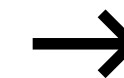
Three.js로 차량(A2Z 자율주행차 ROii) 및 차량의 Zonal 아키텍처로 구성된 3D 모델을 제작하고, model-viewer로 이를 애플리케이션에서 렌더링



A2Z 공식 홈페이지



아키텍처 설계 문서



Three.js 기반 3D 차량 모델

3. 구현 설명

3.3. 차량 환경(AAOS) 최적화 및 고려사항 - 성능 최적화

- GPU 가속 활용: *model-viewer*를 통해 네이티브 수준의 WebGL 하드웨어 가속을 적용하여, 차량용 디스플레이에서도 안정적인 프레임을 확보
- Native-Flutter 분리: 무거운 데이터 처리는 연산 속도가 빠른 안드로이드 네이티브에서, UI 표시는 가벼운 플러터에서 전담하도록 역할을 분리해 부하를 분산
- 선택적 렌더링: *Riverpod*을 이용한 상태 관리를 통해 데이터 변경 시 전체 화면이 아닌 관련 부분만 업데이트 되도록 하여 CPU 점유율을 낮춤

.

3. 구현 설명

3.3. 차량 환경(AAOS) 최적화 및 고려사항 - 운전자 경험 고려 (UX/UI)

- 가시성 확보:
 - 아이콘 점멸 효과로 텍스트 없이 고장 상태 직관적으로 식별가능하도록 함
 - 핫스팟을 통한 레이블링으로 작은 부품의 터치 조작성 및 가시성 확보
- 인지 부하 최소화:
 - 선택적 정보 노출을 적용, 요청 시에만 상세 정보 표시하도록 함
 - 불필요한 요소를 최대한 배제한 UI로 인지부하 줄이고자 함
- 주행 안전 준수:
 - 차량 주행 시 애플리케이션 사용이 불가하도록 함

3. 구현 설명

3.4. 유지보수 및 확장성 전략

다른 차량 및 zonal 아키텍처를 애플리케이션에 적용하고자 할 경우, 다음 두 단계의 작업을 통해 가능하도록 함.

1. 3D 모델 교체
2. 교체한 모델에 맞게 label 데이터 매핑

4. 성능 지표 및 검증 계획

현재는 기능 구현 단계로, 실제 하드웨어 연결을 제외한 주요 기능 개발을 완료했다. 이에 따라 향후 검증 계획을 수립했다.

- a. 기술적 성능 검증: Zonal 네트워크 연결 시 시스템의 응답 속도와 안정성 검증
 - 지연 시간: CoAP 패킷 수신부터 렌더링 반영까지 소요 시간
 - 안정성: 3D 모델 조작 및 고장 시각화 시 평균 FPS 및 메모리 점유율 측정
- b. 사용자 대상 검증: 엔지니어 및 정비사를 대상으로 정성/정량적 평가 진행
 - 정성적 평가 - Olsen's Heuristics 기준 사용자 인터뷰를 통한 시스템 가치 평가
 - 정량적 평가
 - 효율성: 고장 진단 소요 시간, 에러율 측정(기존 응용 및 관제 SW와의 비교)
 - 사용성: SUS (System Usability Scale) 지표
 - 인지 부하: NASA-TLX 지표